

LABORATORIO No. 1

Construcción de una API REST con Spring Boot y MySQL

(Incluye Reto Adicional)

Aprendiz: Luis David Conde Sanchez	Programa: Análisis y Desarrollo de Software (SENA)
Duración Estimada: 2 horas	Entorno: Spring Boot v4.1.0, MySQL, Postman, Java 17

1. Contexto del Laboratorio y Objetivos

Competencia: Desarrollar componentes de software de acuerdo con las especificaciones técnicas y los requerimientos funcionales.

Resultado de aprendizaje: Construir una API REST que permita realizar operaciones CRUD utilizando Spring Boot, MySQL y Postman.

Situación Problema (TechStore S.A.S.):

La empresa TechStore S.A.S. requiere una API REST robusta para la administración de sus usuarios. Como parte del laboratorio, se desarrollará el ejercicio completo integrando directamente el RETO ADICIONAL desde el inicio, lo que incluye los campos extendidos: nombre, correo, teléfono, fechaNacimiento y ciudad.

2. Desarrollo del Proyecto y Registro de Evidencias

A continuación, se disponen los espacios correspondientes para la inserción de las capturas de pantalla de la ejecución del proyecto.

Parte 1 – Configuración e Inicialización del Proyecto

Generación del proyecto en Spring Initializr con las dependencias: Spring Web, Spring Data JPA, MySQL Driver y Validation.



Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.1.1 (SNAPSHOT) ☒ 4.1.0 ☐ 4.0.8 (SNAPSHOT) ☐ 4.0.7

Project Metadata

Group

com.sena

Artifact

laboratorio

Package name

com.sena.laboratorio

Packaging

☒ Jar ☐ War

Configuration

☒ Properties ☐ YAML

Java

☐ 26 ☐ 25 ☐ 21 ☒ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Lombok

DEVELOPER TOOLS

Java annotation library which helps to reduce boilerplate code.

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

MySQL Driver

SQL

MySQL JDBC driver.

HTTP Client

WEB

Spring Boot integration for RestClient and RestTemplate to make HTTP requests.

Spring Data JPA

SQL

Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

Validation

IO

Bean Validation with Hibernate validator.

Parte 2 – Base de Datos y Configuración (application.properties)

Creación de la base de datos 'laboratorio_backend' en MySQL Workbench y estructura de conexión en application.properties.

 EVIDENCIA: Base de Datos y Workbench

```
1 CREATE DATABASE laboratorio_backend;
2
3 USE laboratorio_backend;
4
5 CREATE TABLE usuario(
6     id INT AUTO_INCREMENT PRIMARY KEY,
7     nombre VARCHAR(100),
8     correo VARCHAR(150),
9     telefono VARCHAR(20),
10    fecha_nacimiento DATE,
11    ciudad VARCHAR(100)
12 );
```

Output

Action Output

#	Time	Action	Message
✓ 1	14:45:41	CREATE DATABASE laboratorio_backend	1 row(s) affected
✓ 2	14:45:41	USE laboratorio_backend	0 row(s) affected
✓ 3	14:45:41	CREATE TABLE usuario(id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100), correo VARCHAR(150), telefono VARC...	0 row(s) affected

 EVIDENCIA: application.properties

```
1  server.port=8080
2  spring.application.name=Laboratorio
3
4  spring.datasource.url=jdbc:mysql://localhost:3306/laboratorio_backend
5  spring.datasource.username=root
6  spring.datasource.password=1234
7
8
9  spring.jpa.hibernate.ddl-auto=update
10 spring.jpa.show-sql=true
11
```

Partes 3 a 6 – Implementación del Código Fuente (Arquitectura en Capas)

Capturas de la estructura del proyecto e implementación de las capas (Entidad Usuario con campos extendidos, UsuarioRepository, UsuarioService y UsuarioController).

 EVIDENCIA: Entidad Usuario (Model)

```
1  package com.sena.laboratorio.model;
2  import jakarta.persistence.*;
3  import java.time.LocalDate;
4
5  import lombok.Getter;
6  import lombok.Setter;
7  @Entity
8  @Getter
9  @Setter
10 public class Usuario{
11     @GeneratedValue(strategy=GenerationType.IDENTITY)
12     @Id
13     private Integer id;
14     private String nombre;
15     private String correo;
16     private String telefono;
17     private String ciudad;
18     private LocalDate fechaNacimiento;
19     public Usuario(){ }
20 }
```

EVIDENCIA: UsuarioRepository

```
1 package com.sena.laboratorio.repository;
2 import org.springframework.data.jpa.repository.JpaRepository;
3 import com.sena.laboratorio.model.Usuario;
4 public interface UsuarioRepository extends JpaRepository<Usuario,Integer>{
5
6
7 }
8
```

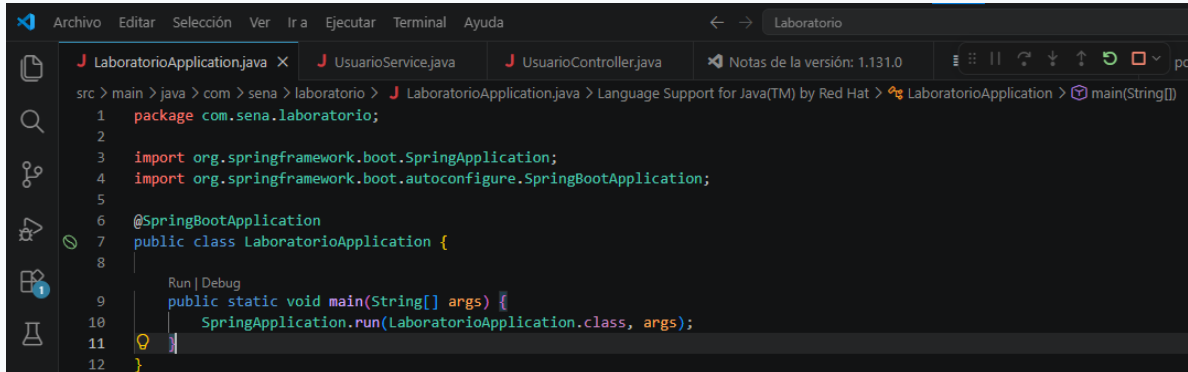
EVIDENCIA: UsuarioService

```
1 package com.sena.laboratorio.service;
2 import org.springframework.stereotype.Service;
3 import java.util.*;
4 import com.sena.laboratorio.model.Usuario;
5 import com.sena.laboratorio.repository.UsuarioRepository;
6 @Service public class UsuarioService{
7     private final UsuarioRepository r;
8     public UsuarioService(UsuarioRepository r){this.r=r;}
9
10    public List<Usuario> listar(){
11        return r.findAll();
12    }
13    public Usuario listarPorId(Integer id){
14        return r.findById(id).orElse(null);
15    }
16    public Usuario guardar(Usuario u){
17        return r.save(u);
18    }
19    public void eliminar(Integer id){
20        r.deleteById(id);
21    }
22 }
```

EVIDENCIA: UsuarioController

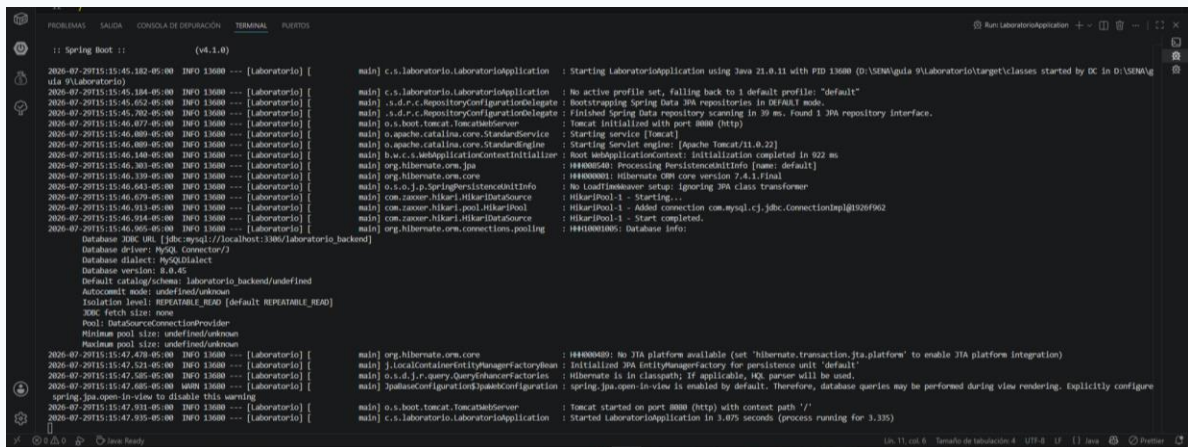
```
1  package com.sena.laboratorio.controller;
2  import org.springframework.web.bind.annotation.*;
3  import java.util.*;
4  import com.sena.laboratorio.model.Usuario;
5  import com.sena.laboratorio.service.UsuarioService;
6  import org.springframework.web.bind.annotation.PutMapping;
7  import org.springframework.web.bind.annotation.RequestBody;
8  import org.springframework.web.bind.annotation.PathVariable;
9
10 @RestController
11 @RequestMapping("/usuarios")
12 public class UsuarioController{
13     private final UsuarioService s;
14     public UsuarioController(UsuarioService s){this.s=s;}
15
16     @GetMapping ("/listar")
17     public List<Usuario> listar(){return s.listar();}
18
19     @GetMapping ("/listar/{id}")
20     public Usuario listarPorId(@PathVariable Integer id){
21         return s.listarPorId(id);
22     }
23
24     @PostMapping ("/guardar")
25     public Usuario guardar(@RequestBody Usuario u){return s.guardar(u);}
26
27     @DeleteMapping ("/eliminar/{id}")
28     public void eliminar(@PathVariable Integer id){s.eliminar(id);}
29
30     @PutMapping("/actualizar/{id}")
31     public Usuario actualizar(@PathVariable Integer id, @RequestBody Usuario u) {
32         u.setId(id);
33         return s.guardar(u);
34     }
35
36 }
```

EVIDENCIA: Proyecto Ejecutándose



```
src > main > java > com > sena > laboratorio > LaboratorioApplication.java > Language Support for Java(TM) by Red Hat > LaboratorioApplication > main(String[] args)

1 package com.sena.laboratorio;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class LaboratorioApplication {
8
9     Run | Debug
10    public static void main(String[] args) {
11        SpringApplication.run(LaboratorioApplication.class, args);
12    }
13 }
```



```
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
Run: LaboratorioApplication

:: Spring Boot :: (v4.1.0)

2026-07-29T15:15:45.582-05:00 INFO 13600 --- [Laboratorio] [
  uia 9/Laboratorio]
2026-07-29T15:15:45.584-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:45.652-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:45.702-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.077-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.080-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.080-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.080-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.140-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.303-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.339-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.643-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.679-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.913-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.954-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:46.965-05:00 INFO 13600 --- [Laboratorio] [

Database JDBC URL [jdbc:mysql://localhost:3306/laboratorio_backend]
Database driver: MySQL Connector/J
Database dialect: MySQLDialect
Database version: 8.0.45
Default catalog/schema: Laboratorio_backend/undefined
Autocommit mode: undefined/unknown
Isolation level: REPEATABLE_READ [default REPEATABLE_READ]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown

2026-07-29T15:15:47.478-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:47.521-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:47.525-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:47.685-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:47.935-05:00 INFO 13600 --- [Laboratorio] [

Spring.jpa.open-in-view to disable this warning
2026-07-29T15:15:47.935-05:00 INFO 13600 --- [Laboratorio] [
2026-07-29T15:15:47.935-05:00 INFO 13600 --- [Laboratorio] [

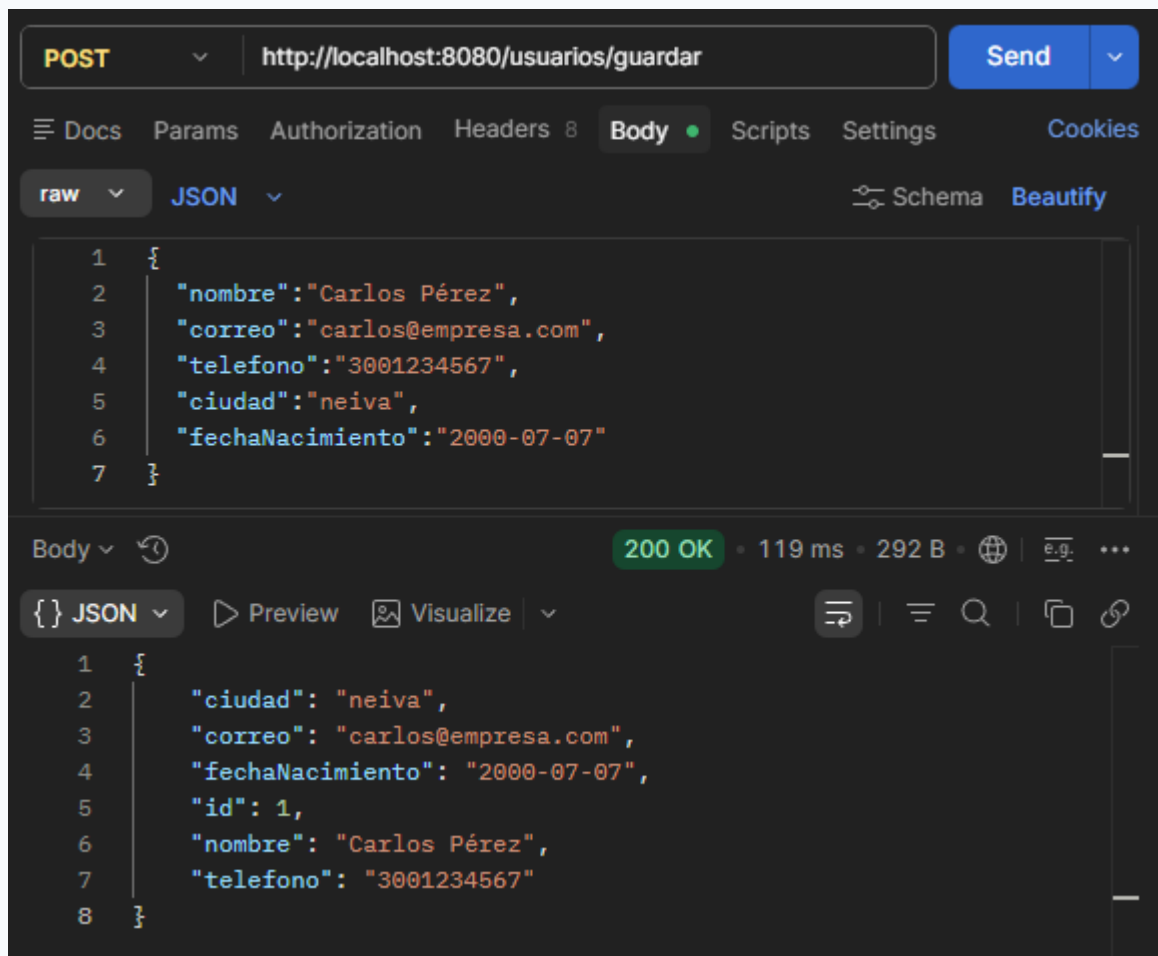
main) c.s.laboratorio.LaboratorioApplication : Starting LaboratorioApplication using Java 21.0.11 with PID 13600 (D:\Sena\gda 9\Laboratorio\target\classes started by DC in D:\Sena\g
main) c.s.laboratorio.LaboratorioApplication : No active profile set, falling back to 1 default profile: "default"
main) s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring data JPA repositories in DEFAULT mode.
main) s.d.r.c.RepositoryConfigurationDelegate : Finished Spring data repository scanning in 39 ms. Found 1 JPA repository interface.
main) o.s.b.t.tomcat.TomcatStarter : Tomcat initialized with port 8080 (http)
main) o.s.b.t.tomcat.TomcatStarter : Starting service [Tomcat]
main) o.s.b.t.tomcat.TomcatStarter : Starting Servlet engine: [Apache Tomcat/11.0.22]
main) h.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 922 ms
main) org.hibernate.orm.jpa : HH000040: Processing PersistenceUnitInfo [name: default]
main) org.hibernate.orm.jpa : HH000000: Hibernate ORM core version 7.4.1.Final
main) o.s.o.j.p.SpringPersistenceUnitInfo : No loadTimeaver setup; ignoring JPA class transformer
main) com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
main) com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@1926f962
main) com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
main) org.hibernate.orm.connections.pooling : HH00001005: Database info:

main) org.hibernate.orm.jpa : HH00000009: No JPA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
main) j.localContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
main) o.s.o.j.r.query.QueryEntityManagerFactory : Hibernate is in classpath; if applicable, H2 parser will be used.
main) hibernate.cfg.Configuration$PersistenceConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure
main) o.s.b.t.tomcat.TomcatStarter : Tomcat started on port 8080 (http) with context path '/'
main) c.s.laboratorio.LaboratorioApplication : Started LaboratorioApplication in 3.875 seconds (process running for 3.335)
```

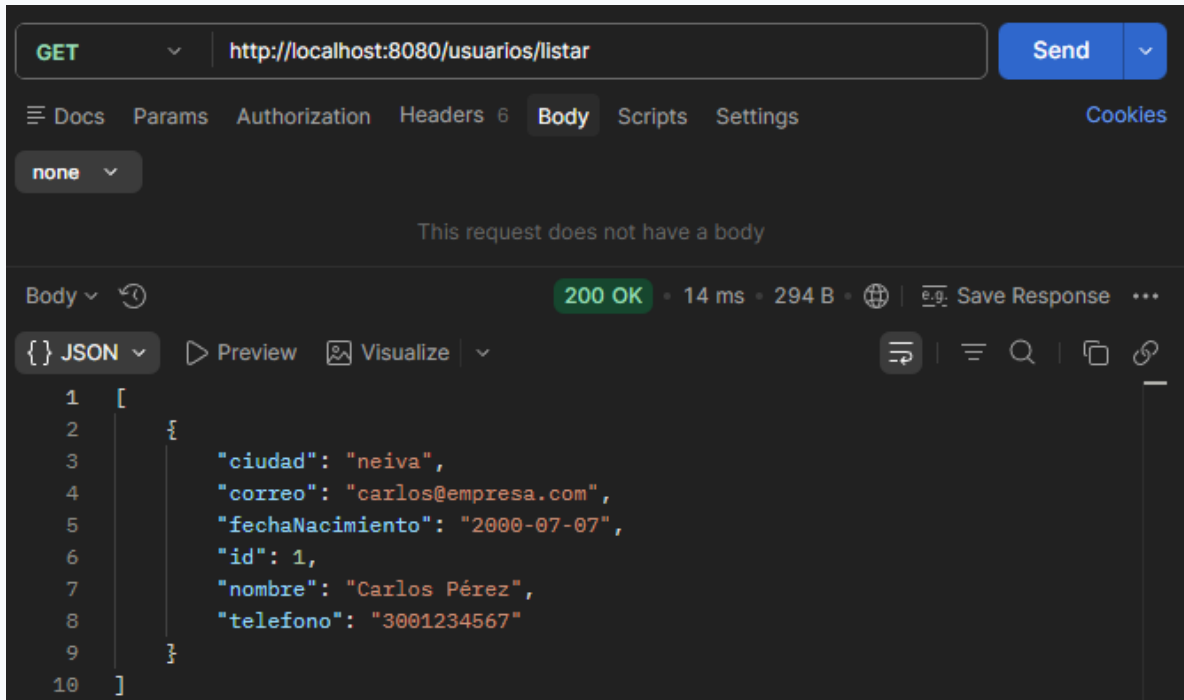
Parte 7 – Pruebas de Endpoints con Postman

Evidencias de la ejecución de pruebas enviando y recibiendo el payload JSON con la estructura completa.

EVIDENCIA: Prueba 1: Registrar Usuarios (POST /usuarios)



EVIDENCIA: Prueba 2: Consultar Todos los Usuarios (GET /usuarios)



GET <http://localhost:8080/usuarios/listar> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

none

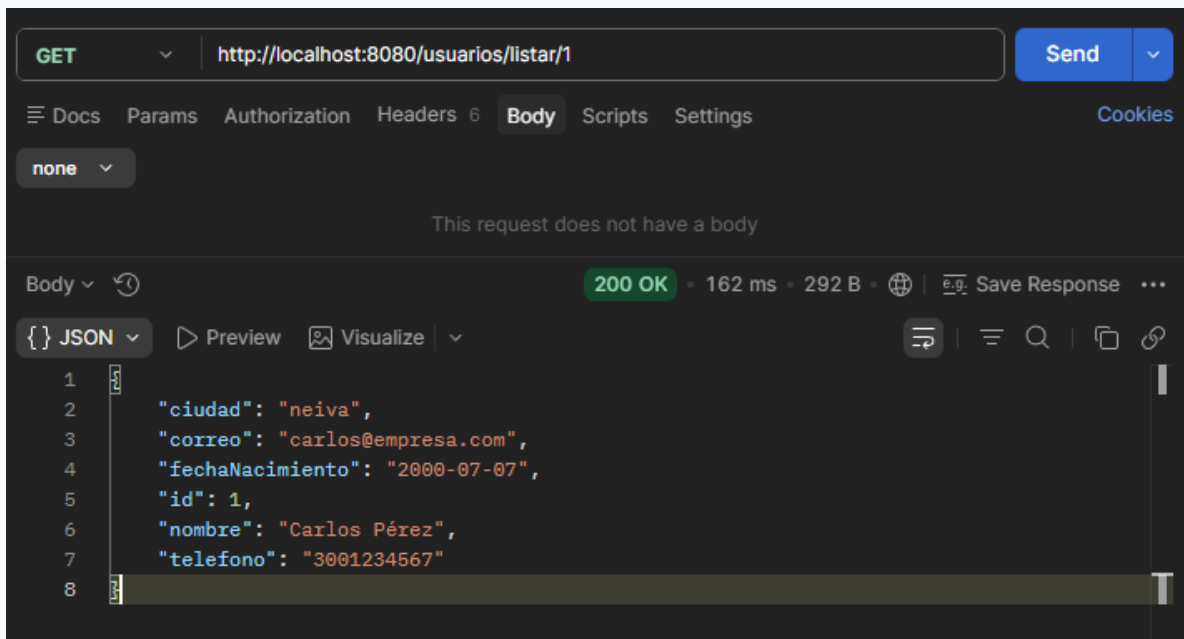
This request does not have a body

Body  200 OK • 14 ms • 294 B •  e.g. Save Response ...

{ } JSON Preview Visualize

```
1  [
2    {
3      "ciudad": "neiva",
4      "correo": "carlos@empresa.com",
5      "fechaNacimiento": "2000-07-07",
6      "id": 1,
7      "nombre": "Carlos Pérez",
8      "telefono": "3001234567"
9    }
10 ]
```

EVIDENCIA: Prueba 3: Consultar Usuario por ID (GET /usuarios/{id})



GET <http://localhost:8080/usuarios/listar/1> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

none

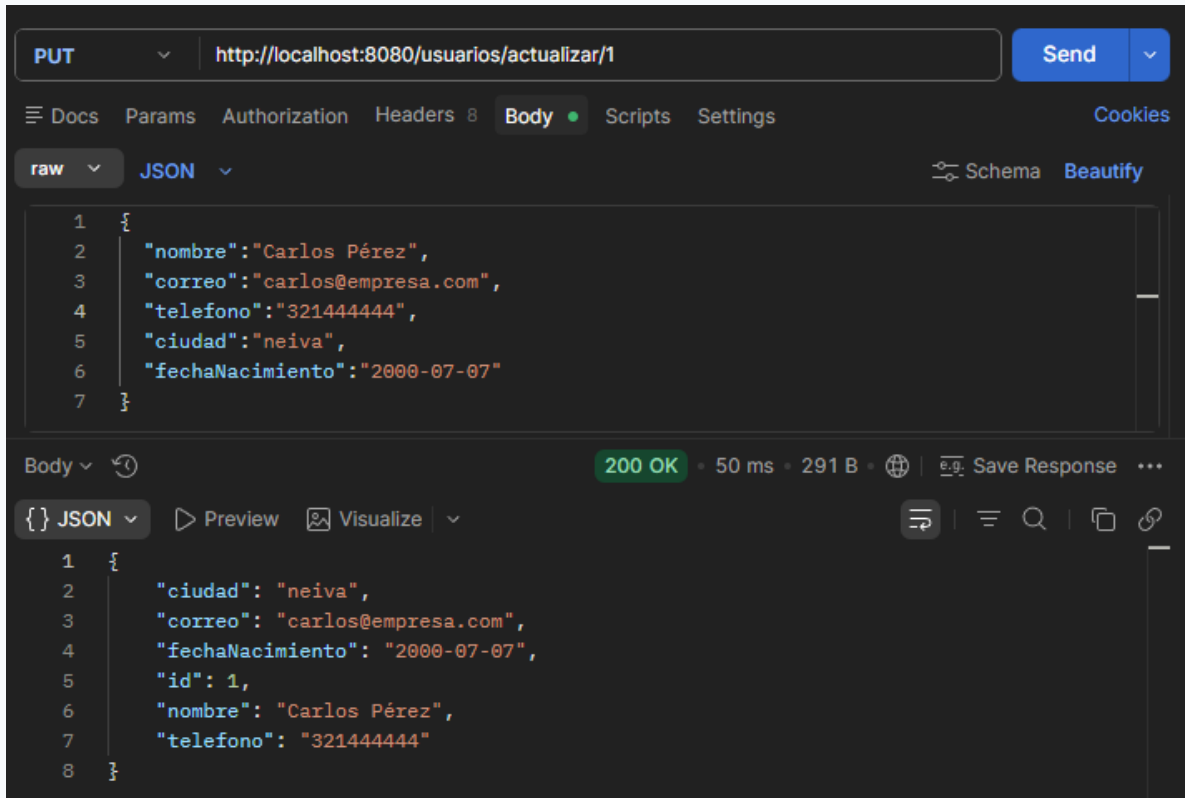
This request does not have a body

Body  200 OK • 162 ms • 292 B •  e.g. Save Response ...

{ } JSON Preview Visualize

```
1  {
2    "ciudad": "neiva",
3    "correo": "carlos@empresa.com",
4    "fechaNacimiento": "2000-07-07",
5    "id": 1,
6    "nombre": "Carlos Pérez",
7    "telefono": "3001234567"
8  }
```


EVIDENCIA: Prueba 4: Actualizar Usuario (PUT /usuarios/{id})



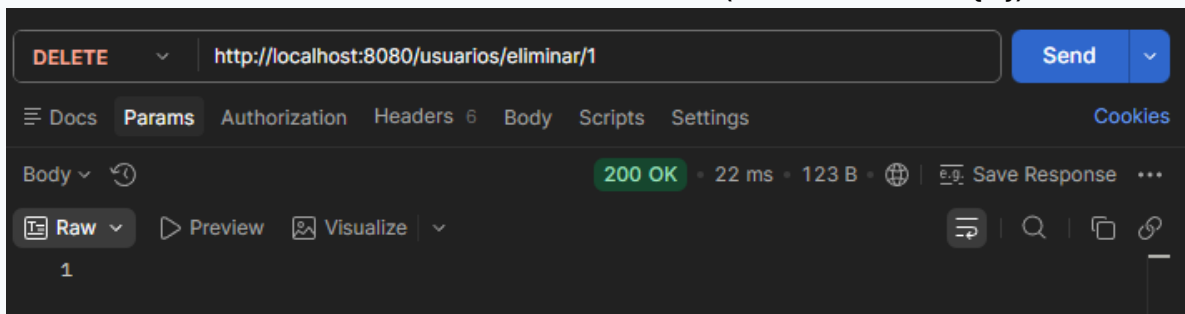
The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** `http://localhost:8080/usuarios/actualizar/1`
- Body:** JSON
- Request Body:**

```
1 {  
2   "nombre": "Carlos Pérez",  
3   "correo": "carlos@empresa.com",  
4   "telefono": "321444444",  
5   "ciudad": "neiva",  
6   "fechaNacimiento": "2000-07-07"  
7 }
```
- Response:** 200 OK, 50 ms, 291 B
- Response Body:** JSON

```
1 {  
2   "ciudad": "neiva",  
3   "correo": "carlos@empresa.com",  
4   "fechaNacimiento": "2000-07-07",  
5   "id": 1,  
6   "nombre": "Carlos Pérez",  
7   "telefono": "321444444"  
8 }
```

EVIDENCIA: Prueba 5: Eliminar Usuario (DELETE /usuarios/{id})



The screenshot shows a REST client interface with the following details:

- Method:** DELETE
- URL:** `http://localhost:8080/usuarios/eliminar/1`
- Body:** Raw
- Response:** 200 OK, 22 ms, 123 B
- Response Body:** Raw

```
1
```

3. Actividad de Análisis y Cuestionario

1. ¿Cuál es la función del Controller?

Es la "puerta de entrada" de la aplicación. Se encarga de recibir las peticiones que hace el usuario desde internet (como enviar datos desde un formulario o pedir información), llama al servicio para procesarlas y devuelve la respuesta adecuada al cliente.

2. ¿Cuál es la función del Service?

Es donde vive la lógica del negocio. Contiene las reglas del Sistema. por ejemplo, validar si los datos están correctos o hacer cálculos. Y también se encarga de coordinar los datos antes de guardarlos o entregarlos.

3. ¿Qué hace el Repository?

Es el encargado de comunicarse directamente con la base de datos. Se encarga de realizar las operaciones básicas de guardar, buscar, actualizar y eliminar registros sin que tengamos que escribir consultas complejas manualmente.

4. ¿Qué diferencia existe entre GET y POST?

- **GET:** Se usa solo para pedir o consultar información del servidor, como ver la lista de usuarios. sin modificar nada.
- **POST:** Se usa para enviar información nueva al servidor con el fin de crear un registro o validar datos, como registrar un nuevo usuario o iniciar sesión.

5. ¿Qué diferencia existe entre PUT y PATCH?

- **PUT:** Se usa para reemplazar un registro completo con nueva información.
- **PATCH:** Se usa para hacer cambios parciales, es decir, modificar solo uno o dos datos específicos sin tocar el resto.

6. ¿Qué significa el código HTTP 404?

Significa "No encontrado" (*Not Found*). Aparece cuando la ruta que intentas consultar no existe o cuando el recurso/ID buscado no está registrado en el sistema.

7. ¿Qué significa el código HTTP 201?

Significa "Creado con éxito" (*Created*). Es la respuesta que nos da el servidor cuando realizamos una petición, como un POST, y el nuevo elemento se guardó correctamente en la base de datos.

8. ¿Por qué JSON es el formato más utilizado en las APIs REST?

Porque es muy liviano, rápido de transferir y fácil de leer tanto para las personas como para las computadoras. Además, se entiende con casi cualquier lenguaje de programación.

9. ¿Qué ventajas ofrece Spring Boot frente al desarrollo tradicional?

Ahorra muchísimo tiempo porque configura casi todo de forma automática, ya trae un servidor web integrado que es Tomcat, para ejecutar el proyecto sin instalaciones extra y evita tener que escribir código repetitivo.

10. ¿Qué ocurriría si la base de datos estuviera fuera de servicio?

La API no podría guardar ni recuperar información. Al intentar hacer una petición, el sistema fallaría al intentar conectarse y respondería con un error del servidor: código HTTP 500 - Internal Server Error.

4. Criterios de Evaluación

Criterio de Evaluación	Puntaje Máximo
Configuración correcta del proyecto (Spring Initializr, dependencias)	10 pts
Conexión a MySQL y archivo application.properties	10 pts
Creación de la entidad Usuario (con atributos del reto adicional)	10 pts
Implementación del Repository (JpaRepository)	10 pts
Implementación de la capa Service (Lógica de negocio CRUD)	15 pts
Implementación del Controller (Endpoints RESTful)	15 pts
Pruebas completas con Postman (Evidencias de capturas)	20 pts
Análisis, respuestas al cuestionario y conclusiones	10 pts

5. Conclusiones del Aprendiz

Considero que este laboratorio nos abre la puerta a todos los aprendices del programa para continuar profundizando en el desarrollo con Spring Boot. En el taller se aprecian de manera clara y práctica los componentes básicos requeridos para estructurar una API REST funcional. Además, la actividad fortalece directamente nuestro lenguaje técnico como desarrolladores profesionales, permitiéndonos comprender el flujo de trabajo real en el diseño de software.